Guía: cómo trabajar en grupo con GitHub

Para alumnos · Proyecto grupal publicado en GitHub Pages · Octubre 2026

Cada grupo arma y administra su propio repositorio en GitHub. Todos los integrantes suben cambios ahí y el proyecto queda publicado en GitHub Pages para que el profesor lo vea funcionando.

Antes de empezar:

- Cada integrante necesita una cuenta de GitHub (gratis, en github.com).
- El grupo elige un **responsable del repositorio**: es quien hace los pasos 1 a 4. Los demás arrancan en el paso 5.
- Tener instalado Git (git-scm.com) o, si prefieren no usar la terminal, GitHub Desktop (desktop.github.com).

Paso 1 — Crear el repositorio (solo el responsable)

1. Entrar a github.com y tocar **New** (o el **+** arriba a la derecha → **New repository**).
2. Nombre: algo claro, por ejemplo `grupo3-proyecto-final`.
3. Elegir **Public**. Es necesario para que GitHub Pages funcione gratis.
4. Tildar **Add a README file**.
5. Tocar **Create repository**.

> Si el proyecto es una página web, el archivo principal tiene que llamarse `index.html` y estar en la raíz del repo.

Paso 2 — Invitar a los compañeros (solo el responsable)

1. En el repo, ir a **Settings → Collaborators** (puede pedir la contraseña de nuevo).
2. Tocar **Add people** y escribir el usuario o mail de cada compañero.
3. Cada compañero recibe un mail o notificación y tiene que tocar **Accept invitation**.

Hasta que no acepten la invitación, no van a poder subir cambios.

Paso 3 — Activar GitHub Pages (solo el responsable)

1. En el repo, ir a **Settings → Pages**.
2. En **Source**, elegir **Deploy from a branch**.
3. En **Branch**, elegir `main` y la carpeta `/ (root)`. Tocar **Save**.
4. Esperar uno o dos minutos y recargar: arriba aparece el link del sitio, con la forma `https://usuario.github.io/nombre-del-repo/`.

Cada vez que se suban cambios a `main`, el sitio se actualiza solo en uno o dos minutos.

Paso 4 — Proteger la rama main (solo el responsable)

Así nadie sube cambios directo a `main` y se evita romper la página publicada.

1. En el repo, ir a **Settings → Branches → Add branch protection rule** (en algunas cuentas aparece como **Add classic branch protection rule** o **Rulesets**).
2. En **Branch name pattern** escribir `main`.
3. Tildar **Require a pull request before merging**.
4. Tildar **Require approvals** y dejar **1**.
5. Tocar **Create** o **Save changes**.

Desde ahora, cada cambio necesita que otro integrante lo apruebe antes de entrar a `main`.

Paso 5 — Descargar el repo en tu compu (todos)

1. En el repo, tocar el botón verde **Code** y copiar el link HTTPS.
2. En la terminal, ir a la carpeta donde quieras guardarlo y ejecutar:

```
git clone https://github.com/usuario/nombre-del-repo.git
cd nombre-del-repo
```

3. La primera vez, configurar tu nombre y mail (los mismos de GitHub):

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-mail@ejemplo.com"
```

Con GitHub Desktop: **File → Clone repository**, elegir el repo de la lista y tocar **Clone**.

¿Cómo se organizan las ramas?

Respuesta corta: una rama por tarea, no por estudiante. Cada tarea es una funcionalidad concreta (el menú, el formulario, la sección de contacto). La hace un estudiante en su rama, la sube con un Pull Request, un compañero la revisa y, una vez integrada, esa rama se borra. Para la próxima tarea se crea una rama nueva.

Por qué no una rama por estudiante

- Una rama que vive todo el cuatrimestre se va alejando de lo que suben los demás y termina llena de conflictos difíciles de resolver.
- Los Pull Requests quedan enormes y nadie los revisa bien.
- No se ve qué tarea está terminada y cuál no.

Cómo se nombran las ramas

Formato: `estudiante/tarea`, en minúsculas y sin espacios. El nombre dice quién la hace y qué hace, así el profesor ve la autoría de un vistazo:

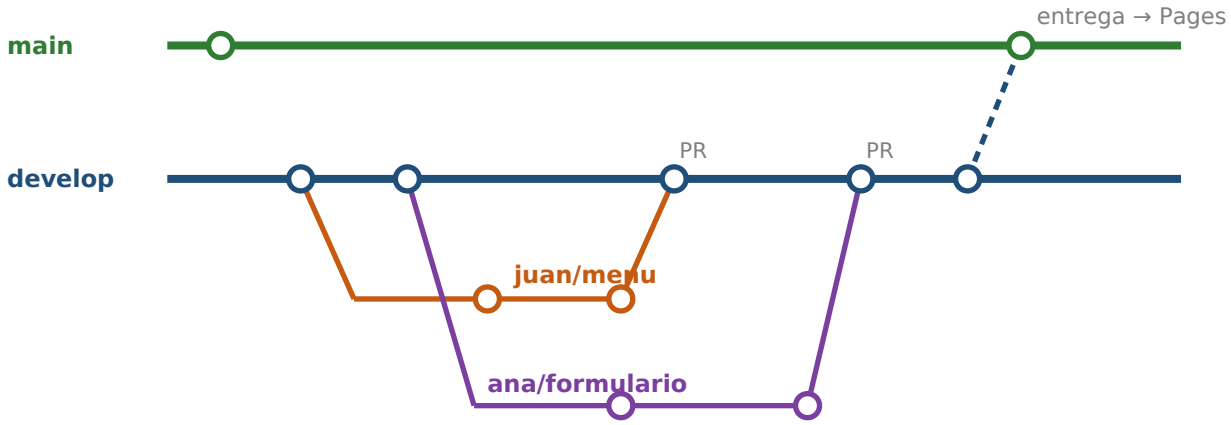
```
juan/menu-navegacion
ana/formulario-contacto
lucas/estilos-pie-de-pagina
```

Las dos formas de trabajar

	Opción A: tareas → main	Opción B: tareas → develop → main
Cómo funciona	Cada rama de tarea se integra directo a main por Pull Request.	Hay una rama auxiliar **develop**. Las tareas se integran ahí; cuando todo funciona junto, se pasa develop a main.
Qué se ve en Pages	Cada cambio aprobado se publica enseguida.	Solo se publica cuando el grupo decide hacer una entrega.
Ventaja	Más simple: menos pasos y menos ramas.	Main siempre tiene una versión estable; se pueden probar las tareas juntas antes de publicar.
Cuándo conviene	Grupos de 2 a 4, proyectos chicos, primeras veces con Git.	Grupos más grandes o entregas por etapas (por ejemplo, entrega 1, entrega 2).

Recomendación: empezar con la Opción A. Es lo que ya explican los pasos 6 y 7. Pasar a la Opción B si el grupo es grande o si se quiere que la página publicada solo cambie en cada entrega.

Esquema de la Opción B



Cada estudiante saca su rama desde develop, la integra a develop con un PR y, en cada entrega, el responsable abre un PR de develop a main.

Cómo armar la Opción B (solo el responsable, una vez)

```
git checkout main
git pull
git checkout -b develop
git push -u origin develop
```

- En **Settings → Branches**, agregar otra regla igual a la del Paso 4 pero con el patrón `develop`.
- En **Settings → General → Default branch**, cambiar a `develop`: así los Pull Requests apuntan ahí por defecto.
- En el Paso 6, reemplazar `main` por `develop` al hacer `git checkout` y `git pull`.
- Para cada entrega: en GitHub, **New pull request** con base `main` y compare `develop`, aprobar y hacer merge.

Paso 6 — Trabajar y subir cambios (todos, cada vez)

Cada tarea se hace en una rama propia. Nunca trabajar directo en `main`.

1 Traer lo último que subieron los demás:

```
git checkout main
git pull
```

2 Crear una rama para tu tarea (formato estudiante/tarea):

```
git checkout -b juan/menu-navegacion
```

3 Hacer los cambios. Cuando tengas algo que funciona, guardarlo:

```
git add .
git commit -m "Agrego el menú de navegación"
```

4 Subir la rama a GitHub:

```
git push -u origin juan/menu-navegacion
```

5 Entrar al repo en GitHub: aparece un aviso amarillo con el botón **Compare & pull request**. Tocarlo, escribir qué cambiaste y tocar **Create pull request**.

6 Avisarle al grupo que hay un Pull Request para revisar.

Con GitHub Desktop: **Current branch → New branch**, después **Commit to …**, **Publish branch** y **Create Pull Request**.

Paso 7 — Revisar y aprobar el cambio de un compañero

Nadie puede aprobar su propio Pull Request: lo aprueba otro integrante.

1 En el repo, abrir la pestaña **Pull requests** y entrar al de tu compañero.

2 En **Files changed** ver qué modificó. Si algo está mal, dejar un comentario en la línea.

3 Si está bien: **Review changes → Approve → Submit review**.

4 Volver a **Conversation** y tocar **Merge pull request → Confirm merge**.

5 Tocar **Delete branch**: la tarea terminó y esa rama ya no se usa.

En uno o dos minutos el cambio ya se ve en la página de GitHub Pages.

Problemas comunes

Problema	Qué hacer
Al hacer `push` pide contraseña y no la acepta	GitHub no acepta la contraseña de la cuenta. Iniciar sesión desde la ventana del navegador que abre Git, o usar GitHub Desktop.
`Permission denied` o `403` al hacer `push`	Todavía no aceptaste la invitación del Paso 2. Revisá el mail o github.com/notifications.
El `push` a `main` es rechazado	Está bien: main está protegida. Creá una rama con `git checkout -b nombre/tarea` y subí esa (Paso 6).
El Pull Request dice que hay conflictos	Dos personas cambiaron las mismas líneas. Ver abajo.
La página de Pages da error 404	Revisar que exista `index.html` en la raíz y esperar unos minutos después del merge.
No veo los cambios en Pages	Recargar con Ctrl + F5. Revisar en la pestaña **Actions** que el despliegue terminó en verde.

Cómo resolver un conflicto: en tu rama, ejecutar `git pull origin main` (o `develop` si usan la Opción B). Git marca los archivos en conflicto con `<<<<<<<`, `=======` y `>>>>>>>`. Dejar el código correcto, borrar esas marcas y después:

```
git add .
git commit -m "Resuelvo conflicto con main"
git push
```

Para evitarlos: hacer `git pull` antes de crear cada rama, mantener las tareas chicas (que duren días, no semanas) y repartir el trabajo para no tocar el mismo archivo a la vez.

Qué enviarle al profesor

Una vez que el sitio está publicado, el responsable envía un solo mensaje por grupo con:

- Nombre y apellido de cada integrante, con su usuario de GitHub.
- El link del repositorio: `https://github.com/usuario/nombre-del-repo`
- El link de la página publicada: `https://usuario.github.io/nombre-del-repo/`

El profesor ve los aportes de cada integrante en **Insights → Contributors** y en los Pull Requests. Por eso cada uno tiene que subir sus cambios desde su propia cuenta, no desde la compu de un compañero.